ReOrder IQ: Project Report

Oct 29, 2026 · @Utsav

All figures in this report come from synthetic, illustrative data. Customers, orders, campaign delivery and post-campaign reorders were generated for this project; nothing is real UAE marketplace data.

Executive summary

In a 14-day simulated test, reminding declining frequent customers raised the reorder rate from 12.85% to 21.48%, an 8.63-point lift, at a message cost of about 7 AED. The 95% interval on that lift is +0.41 to +16.84 points, so the direction is positive but the evidence is moderate, not conclusive.

ReOrder IQ is a retention agent built in n8n, PostgreSQL and Gemini. It finds customers whose ordering is slowing, applies a fixed policy (reminder, no discount), splits eligible customers into treatment and control, simulates delivery, measures incremental contribution against control, and asks Gemini to explain the result. A human approves every action.

- **Headline lift:** 21.48% treatment vs 12.85% control reorder rate (149 vs 179 customers), about 12.9 extra orders.
- **Incremental contribution:** 8.23 AED per treated customer (95% interval 0.05 to 16.41), about 1,219 AED net after 7.45 AED of message cost.
- **Recommendation:** run a larger confirmation test before scaling. The lower bound of both intervals sits barely above zero.

Business problem and objectives

A food-delivery marketplace loses profit when frequent customers quietly order less, and blanket discounts to win them back often pay customers who would have returned anyway. The scenario is a fictional UAE marketplace with markets in Dubai, Abu Dhabi and Sharjah.

The project had four objectives:

- Detect customers whose recent ordering has dropped sharply against their own history.

- Choose the next action with deterministic rules, not an AI guess, and never discount by default.
- Measure whether the action caused extra orders, using a control group, and report uncertainty.
- Keep a human in the loop: nothing is sent or scaled without approval.

Architecture and data

Four n8n workflows run on a local Docker setup, reading and writing one PostgreSQL database (`reorderiq`). Gemini is used only to explain results, never to compute them.

Workflow	What it does	Writes to
WF-01 Data and opportunity detection	Computes per-customer order metrics, flags at-risk customers, applies policy v1	`action_recommendations`
WF-03 Approval and assignment	Assigns approved customers to treatment or control by a stable hash, then logs sandbox delivery to treatment only	`experiment_assignments` , `campaign_events`
WF-04 Measure results	Compares reorder rate and contribution between groups with 95% intervals	`experiment_results`
WF-05 AI brief	Sends aggregate results to Gemini and stores a structured JSON recommendation	`ai_briefs`

The synthetic dataset has 5,000 customers and 63,616 orders (including 55 simulated post-campaign orders), with about 10% of customers opted out of marketing. About 20% of customers had their order rate cut by 75% in the last 60 days, which is the signal the detector looks for.

Policy v1 (deterministic): a customer is at risk if they placed at least 6 orders in days 61 to 120 and at most half as many in the last 60 days. Customers without marketing consent are a hard stop. Anyone who ordered in the last 7 days is suppressed. Everyone else gets a

personalized reminder with a 0% discount.

Experiment design and method

The test compares customers who received the reminder with a held-out control group who received nothing, so any difference in reordering can be attributed to the reminder.

Step	Rule
Eligibility	At-risk, opted in, policy action is personalized reminder, and approved by a human: 328 customers
Assignment	Stable and mutually exclusive: an MD5 hash of the experiment and customer ID decides the group, with a unique constraint so a customer can never be assigned twice
Treatment	149 customers receive the sandbox reminder (no real messages are sent)
Control	179 customers receive nothing
Outcome window	Completed orders after the campaign start date, 14 days
Contribution	Gross order value minus discount minus variable cost, summed per customer, zeros included
Uncertainty	Normal-approximation 95% intervals for the difference in reorder rates and in contribution per customer
Cost	Assumed 0.05 AED per message sent

The split came out 55% control and 45% treatment. That gap is within normal random variation for 328 customers and is not a sign of bias in the hash.

Results

Treatment customers reordered 8.63 points more often than control, but the 95% range starts at +0.41, so a true effect near zero cannot be ruled out. These are simulated outcomes with a built-in effect, so the result demonstrates the measurement method, not a real-world finding.

Measure	Control	Treatment
Customers	179	149
Reorder rate, 14 days	12.85%	21.48%
Contribution per customer (AED)	12.85	21.08

Incremental measure	Estimate	95% interval
Reorder-rate lift (percentage points)	+8.63	+0.41 to +16.84
Contribution per treated customer (AED)	+8.23	+0.05 to +16.41
Incremental orders	about 12.9	not computed
Message cost (AED)	7.45	assumed 0.05 per send
Net incremental contribution (AED)	1,219.13	not computed

Net contribution is the per-customer difference times 149 treated customers, minus message cost. Because the discount is 0%, there is no discount cost to subtract.

AI brief and governance

Gemini received only aggregate experiment results, never customer-level data, and returned a structured JSON brief that was stored with `pending_approval` status. It recommended seeking human approval before expanding the non-discount reminder campaign, with a message cost of 7.45 AED and guardrails on campaign cost, opt-out rate and net incremental contribution.

The AI rated its own confidence "high", which the statistics do not support. The lift interval starts at +0.41 points and the groups have about 150 customers each, so moderate is the fair rating. The deterministic SQL numbers take precedence over the AI's wording,

which is the point of separating calculation from explanation.

Controls built into the system:

- **Consent hard stop:** customers without marketing consent can never receive an action.
- **No default discount:** the policy recommends reminders only, so no discount cost is incurred.
- **Human approval:** recommendations start as `pending_approval`, and assignment only includes approved ones.
- **No duplicate assignment or sends:** unique constraints and `ON CONFLICT DO NOTHING` make workflow reruns safe.
- **Fail-safe AI step:** the AI explanation can fail or be skipped without blocking the pipeline.

Limitations

The result shows that the pipeline measures an effect correctly. It does not show that reminders work in a real marketplace.

- **Synthetic data:** customers, orders, deliveries, opens, clicks and post-campaign reorders are all generated. The 12% baseline and 17% treated reorder rates were assumptions built into the simulation.
- **Small sample:** about 150 and 180 customers per group give wide intervals; the lower bounds of both are barely above zero.
- **Short window:** 14 days only. Long-term retention, repeat purchase and opt-outs are not measured.
- **Assumed cost:** 0.05 AED per message is an assumption, not a vendor price.
- **Normal-approximation intervals:** adequate at this size but approximate, and contribution per customer is highly skewed.
- **Contribution per order looks high:** simulated orders average roughly 100 AED of contribution each, which should be checked against the order-value assumptions used when the data was generated.
- **Manual approval:** approval was done with an SQL update, not a real approval form.
- **Per-customer AI explanations skipped:** free-tier rate limits made them unreliable, so the AI works at aggregate level only.

Recommendations and next steps

Scale only after a larger confirmation test, because the current evidence is positive but moderate.

1. **Confirmation test:** repeat with at least several hundred customers per group, and a pre-set stopping rule, before any rollout.
2. **Keep the 0% discount policy** unless a separate test shows a discount adds profit beyond what the reminder already earns.
3. **Add an approval form** in n8n so approval is recorded by a named person, not an SQL update.
4. **Track guardrails:** opt-out rate, complaint rate and campaign cost, alongside contribution.
5. **Extend the window** to 30 or 60 days to see whether reordered customers stay active.
6. **Replace assumptions** (message cost, order margin) with measured values from real data.

Appendix: README and data dictionary

Stack: n8n (Docker, localhost:5678), PostgreSQL 16 in Docker (`reorderiq-db`, host port 5433), DBeaver as SQL client, Google Gemini for the explanation step. All data is synthetic.

To reproduce:

1. Start Docker Desktop and both containers (`reorderiq-db` and `n8n`).
2. Create the tables below and load the synthetic customers and orders.
3. Run WF-01, then approve the recommendations you want to test.
4. Run WF-03 once to assign groups and log sandbox delivery.
5. Generate simulated outcomes once, then run WF-04 to measure results.
6. Run WF-05 to produce the AI brief, then read `v_reorderiq_summary`.

Table or view	Purpose
`customers`	5,000 synthetic customers with market, consent status and loyalty tier
`orders`	63,616 orders with gross value, discounts, variable cost and status

Table or view	Purpose
`action_recommendations`	449 at-risk customers with policy action, reason, approval status
`experiment_assignments`	328 stable treatment and control assignments for EXP-001
`campaigns`	Campaign definition CMP-001, sandbox channel, 0% offer
`campaign_events`	Sent, delivered, opened and clicked events for treatment customers
`experiment_results`	One row per measurement run, with lift, intervals and net contribution
`ai_briefs`	Structured Gemini brief stored with approval status
`v_reorderiq_summary`	One-row view joining the latest results and AI brief